AI ACADEMY · LEVEL 5 · GRADE 10 · WEEK 17 · STUDENT LESSON

Initialisation and reproducibility

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

THE CONNECTED WEEK

Save the start. Keep the protocol. Check the whole trace.

Compare initialization paths and hidden-unit symmetry with a complete reproducibility record.

Keep one habit: Complete scalar starts → separate random sequence → pictured equal-distance paths → four-unit symmetry and extra-draw programs → independent new objective.

MAIN LESSON · SECTION 1

What changed at the start,and what does a replay require?

> Save actual parameter values and the full protocol,not only a seed or final score.

Learning goals

I can trace how starting values affect a fixed **training** budget.

I can explain a simple hidden-unit **symmetry**.

I can record enough information to replay and audit a classroom experiment.

Retrieve the prior controlled procedure

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Regularisation and early stopping. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Learning rate experiments. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Updating parameters. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Why must an early-stopping report identify both the stop epoch and the restored checkpoint?

> **My starting explanation**

End goal

By the end, I can explain **initialisation** and reproducibility using a traced example and a stated limitation.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

This week you will practise:

Trace how starting values affect a fixed **training** budget

Explain a simple hidden-unit **symmetry**

Record enough information to replay and audit a classroom experiment

Retrieve what you already know

This week's end goal

MAIN LESSON · SECTION 2

Change only the initial output weight

> Same frozen hidden unit,two training pairs,mean objective,rate and two-update budget.

Complete fixed protocol and recurrence

Return to h = ReLU(x), y = vh, **training** pairs (1, 2) and (2, 4), ordinary MSE and a frozen hidden unit. Only v is trained. Keep full-data updates, learning rate 0.1 and exactly two updates. Compare initial v values 0, 1 and 3.

This experiment changes only the start.

The gradient is 5(v - 2), so each step gives new v = 0.5v + 1. Signed distance from 2 halves on every update.

Different starting distances therefore produce different losses after this finite budget, even though all three runs approach the same minimum if this exact procedure continues.

All three starting-value traces

Initial v	After one update	After two updates	Final training MSE
0	1	1.5	0.625
1	1.5	1.75	0.15625
3	2.5	2.25	0.15625

Equal distance and unequal final models

The equal losses for starts 1 and 3 arise because they are equally distant from 2. They do not have equal final parameters.

This simple convex example does not show the multiple solutions or inactive units that can occur in larger networks.

Name given and missing facts

List the given facts and one fact that is not supplied.

Read the complete scalar traces and the separate random-draw example

> A random sequence,parameter assignment and training protocol are different parts of the record.

Main saved starts,states and predictions

Main initial v	Three saved states(0, 1, 2 updates)	Final training MSE $2.5(v-2)^2$	Final prediction at x1
0	[0, 1, 1.5]	0.625	1.5
1	[1, 1.5, 1.75]	0.15625	1.75
3	[3, 2.5, 2.25]	0.15625	2.25

Pseudorandom sequence and explicit separation

A pseudorandom generator produces a sequence determined by its state. In the same Python setup, recreating a local generator with the same **seed** and making the same calls reproduces the sequence below.

Saving the generated values also removes ambiguity about which draw became which parameter.

The code contains two separate demonstrations. Four uniform random draws from a local seeded generator show repeatability of that sequence in the same setup with the same calls. The three scalar training runs instead start at the explicit values 0, 1, 3; they do not use those draws. The uniform range−1to1is a classroom illustration, not an initialization recommendation.

Identical seed plus different draw order can assign different values to weights. Save actual values, their names/order and the environment as well as the seed.

The four random numbers illustrate a saved **initialisation** sequence; they are not used in the three separate scalar **training** runs.

The range [-1, 1] is a classroom choice, not a general recommendation for network **initialisation**. An extra random draw before assigning weights shifts which value is used, even if the **seed** stays the same.

Complete runnable sequence and scalar-training comparison

```
from random import Random

def initial_values(seed):
    rng = Random(seed)
    return [rng.uniform(-1.0, 1.0) for _ in range(4)]

def train(v):
    states = [v]
    for _ in range(2):
        v = v - 0.1 * 5 * (v - 2)
        states.append(v)
    return states

first = initial_values(17)
second = initial_values(17)
assert first == second
print(first)
for start in [0.0, 1.0, 3.0]:
    print(start, train(start))
```

Connect the picture and the saved starting values

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

Connect the picture and the explanation

Compare equal losses without declaring identical models

> Keep all starting values and paths in the report.

Current equal-loss task

- The lesson runs starting at 1 and 3 finish with the same **training** MSE. Give their final v values and explain the equality without calling the models identical.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured exact replay

- Change: Repeat the a = 0 run with identical arithmetic and settings.

▼ **Check after predicting**

Prediction: It again reaches 1 then 1.5 in this deterministic toy calculation.

Reason: Identical initial state and update rules determine the same arithmetic trace.

▼ **Earlier wording, alternatives and picture transcripts**

Compare a familiar case with a changed case

Change one thing in the pictured case

Trace different scalar starts,then inspect identical hidden units

> Name the exact formula before interpreting a matching result.

Complete pictured scalar facts and separate scope

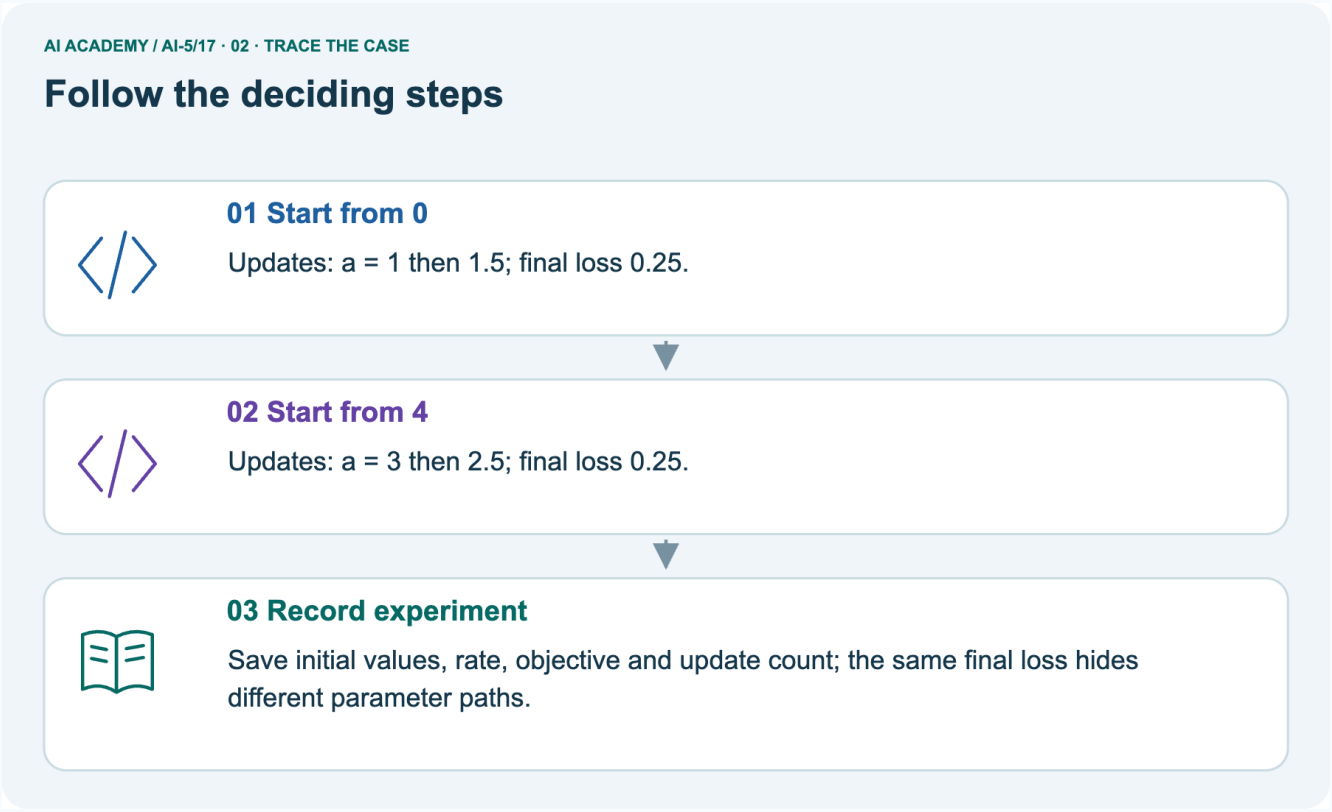
Reproduce two **training** traces by saving their distinct initial values.

Loss J(a)=(a-2)²; gradient 2(a-2); rate 0.25; two updates. Restart from a = 0 and separately a = 4.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

The picture uses a separate scalar objective J(a)=(a−2)², gradient 2(a−2), rate.25 and two updates. Start0 gives0→1→1.5; start 4 gives4→3→2.5; both final losses.25 because distances from 2 are equal. Repeating start 0 with identical arithmetic and settings repeats its path. The main MSEobjective2.5(v−2)²uses rate.1 and starts 0/1/3. The two-unit symmetry extension below is a third, explicitly defined computation.

Trace both scalar starts and equal distances



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Start from 0	Updates: a = 1 then 1.5; final loss 0.25.
2. Start from 4	Updates: a = 3 then 2.5; final loss 0.25.
3. Record experiment	Save initial values, rate, objective and update count; the same final loss hides different parameter paths.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

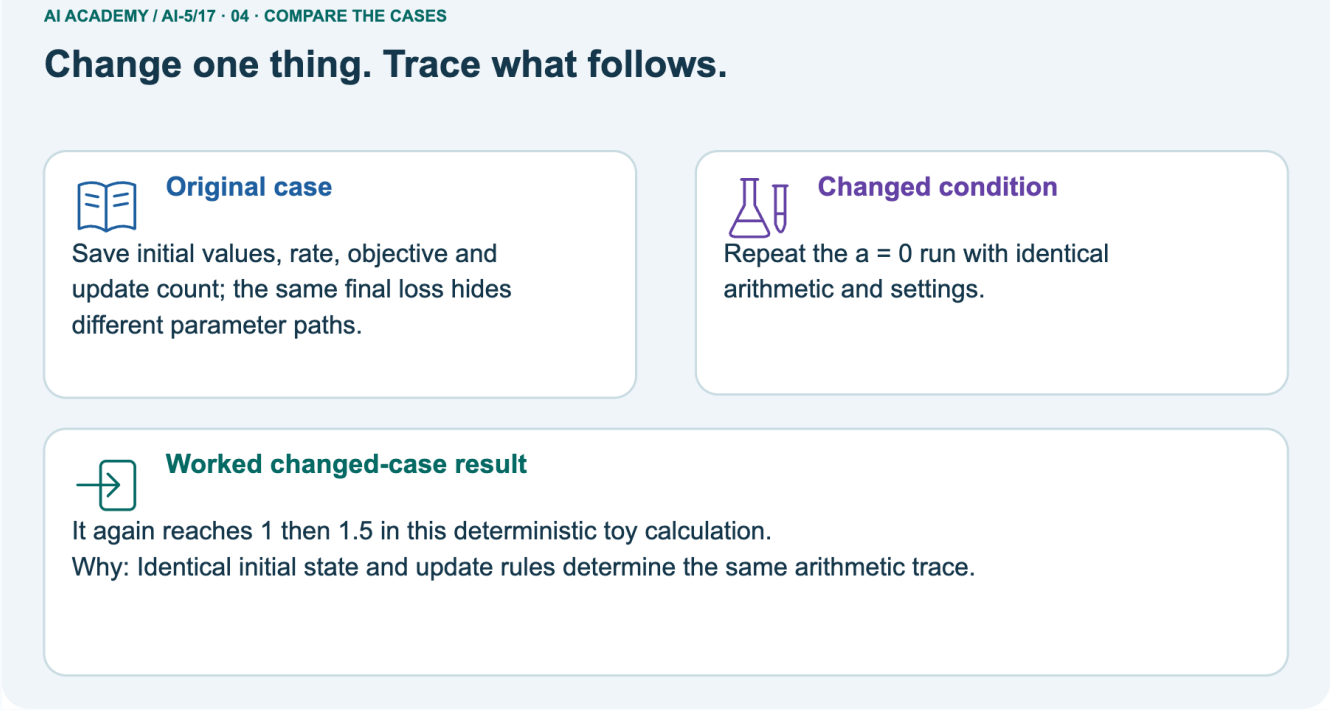
Initialisation supplies the parameter values before learning begins. The same update rule can follow different paths from different starts.

Here the two paths end on opposite sides of two and have equal squared distance, so equal loss does not imply equal parameters. Save initial values and settings to reproduce this arithmetic, as well as the final result.

Predict the identical restart

- Change one thing: Repeat the a = 0 run with identical arithmetic and settings. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain replay limits and matching losses

- A saved **seed** alone does not reproduce every software or hardware execution; record environment and nondeterminism too.

- Read the labelled picture: Does the matching final loss mean the parameter values are equal? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Complete givens for the two-unit symmetry extension

For the symmetry extension, use y=aReLU(ux)+bReLU(vx), parameter order[u, v, a, b], x1, target 1, all four original parameters 1 and rate.1. Both hidden scores and activations are 1, y2, residual 1. Each old-state squared-loss gradient is2, so one simultaneous step makes every parameter.8. New hidden activations are[.8, .8], y=.8×.8+.8×.8=1.28, and loss(.28)²=.0784. The units still match.

Here u and v are input weights and a and b are output weights; these names are separate from the main single trained output v.

Trace and explain the two-unit update

Identical units receive identical updates

HIDDEN UNIT	BEFORE THE UPDATE	AFTER THE UPDATE
First ReLU unit	u is 1 and a is 1	u is 0.8 and a is 0.8
Second ReLU unit	v is 1 and b is 1	v is 0.8 and b is 0.8

In this separate two-unit example there are no biases. At x = 1, output is 2 and target is 1. Each of the four squared-error partial gradients equals 2. A simultaneous update at rate 0.1 changes every weight from 1 to 0.8 and keeps the units identical.

For x = 1, calculate the two-unit output and squared error before and after the shown simultaneous update. Have the units become different?

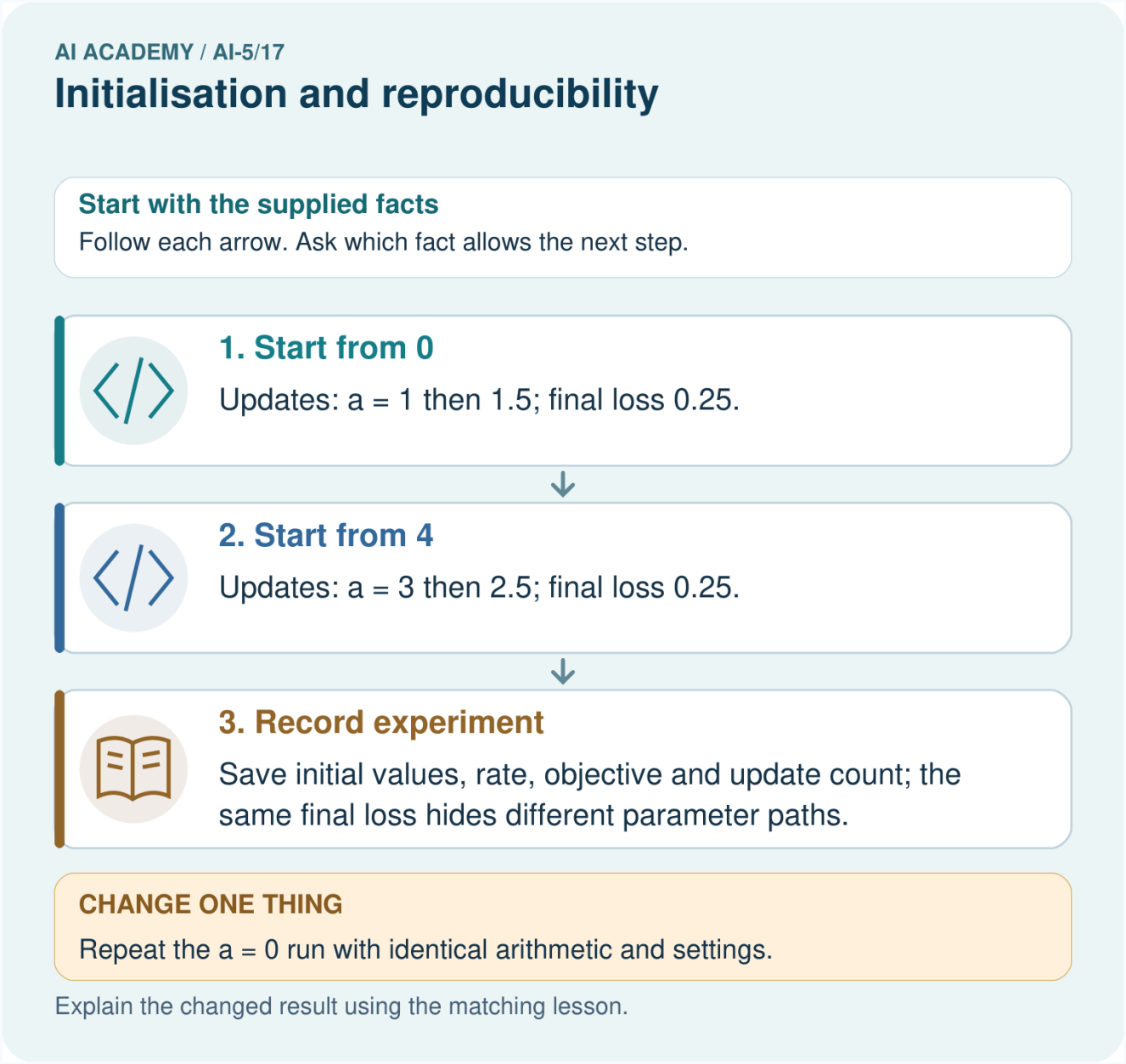
▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Initialisation and reproducibility

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Repair seed-only records and identical-unit assumptions

> A controlled starting-value comparison is not a guarantee for every architecture.

Complete two-unit gradients,symmetry and all-zero limitation

For the two-unit example, y = a ReLU(ux) + b ReLU(vx). At x = 1, u = v = a = b = 1, **output** is 2 and target is 1. Because both ReLUs are in their positive region, changing any one parameter slightly changes y at rate 1.

Squared-error gradient is twice residual times that sensitivity, so each gradient is 2. Students can check it with the symmetric numerical probe from Week 10.

With identical initial weights, identical data and deterministic symmetric updates, these units remain copies and cannot learn distinct roles through those updates alone. Different hidden weights can break the **symmetry**.

For example, u = 1 and v = -1 give hidden activations [1, 0] at x = 1 and [0, 1] at x = -1. This demonstrates different responses, not a guarantee of successful **training**.

Zero is not always an invalid start: v = 0 worked in our single trained-**output**-weight model.

But setting all four weights to zero in the two-unit product model leaves every single-coordinate loss probe unchanged, because each path still contains a zero factor.

Do not generalise the success of one scalar start to every architecture.

Try the extra-preview-draw investigation

- Both runs use the same **seed** and generator. Run A assigns the first four draws to its weights. Run B draws a preview number first, then assigns the next four. Must the initial weight arrays match? What should be saved?

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise start,path,loss and symmetry reasoning with less help

> A replay should preserve intermediate evidence as well as its final score.

Current task and response

- For the scalar lesson model, start v = -2 and apply two updates at rate 0.1. Give all three v values and the final **training** MSE.

For x = 1, calculate the two-unit output and squared error before and after the shown simultaneous update. Have the units become different?

> **Explain your prediction to a partner before comparing results.**

Supported W2record

Use workbook W2 for the connected example “**Initialisation** and reproducibility”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual misconception check

- Use the worked case for **Initialisation** and reproducibility. Proposed explanation: “The same **seed** specifies the entire experiment”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Pictured equal-loss reasoning

- Does the matching final loss mean the parameter values are equal? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

My explanation using a pictured step and the given facts

Reserve the independent new objective

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Distinguish equality of loss and parameters

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Run complete start,random-call and symmetry experiments

> Each runnable example declares whether starts are explicit,drawn or identical.

Current task and complete main sequence/scalar program

The lesson runs starting at 1 and 3 finish with the same **training** MSE. Give their final v values and explain the equality without calling the models identical.

```
from random import Random

def initial_values(seed):
    rng = Random(seed)
    return [rng.uniform(-1.0, 1.0) for _ in range(4)]
```

```
def train(v):
    states = [v]
    for _ in range(2):
        v = v - 0.1 * 5 * (v - 2)
        states.append(v)
    return states

first = initial_values(17)
second = initial_values(17)
assert first == second
print(first)
for start in [0.0, 1.0, 3.0]:
    print(start, train(start))
```

Complete extra-draw assignment comparison

```
from random import Random

def assigned_weights(seed, preview=False):
    rng = Random(seed)
    if preview:
        rng.uniform(-1, 1)  # Same generator call consumes the first draw.
    return [rng.uniform(-1, 1) for _ in range(4)]

normal = assigned_weights(17)
replay = assigned_weights(17)
shifted = assigned_weights(17, preview=True)
print('identical calls repeat', normal == replay)
print('extra preview keeps assigned weights equal', normal == shifted)
print('normal assigned values', normal)
print('shifted assigned values', shifted)
# Save exact arrays and parameter assignment order in the experiment record.
```

Complete four-parameter symmetry experiment

```
# Separate two-unit symmetry example, not the scalar-v training experiment.
# Parameters are in fixed order [u, v, a, b]. No random initialization is used here.
def output(params, x):
    u, v, a, b = params
    return a * max(0, u*x) + b * max(0, v*x)

def update(params, x, target, rate):
    u, v, a, b = params
```

```
    z1, z2 = u*x, v*x
    h1, h2 = max(0,z1), max(0,z2)
    residual = a*h1 + b*h2 - target
    # ReLU derivative convention at zero is 0 for this teaching program.
    gradients = [2*residual*a*x*(z1>0), 2*residual*b*x*(z2>0),
                 2*residual*h1, 2*residual*h2]
    new = [old-rate*g for old,g in zip(params,gradients)]
    return gradients, new, (output(new,x)-target)**2

old = [1.,1.,1.,1.]
gradients, new, loss = update(old, 1, 1, .1)
print('old [u,v,a,b]', old, 'output', output(old,1))
print('gradients', gradients)
print('new [u,v,a,b]', new)
print('new output/loss', round(output(new,1),6), round(loss,6))
print('identical units remain identical', new[0]==new[1] and new[2]==new[3])
print('all-zero start gradients', update([0.,0.,0.,0.],1,1,.1)[0])
print('hidden activations u1/v-1 at x1', [max(0,u*1) for u in [1,-1]])
print('hidden activations u1/v-1 at x-1', [max(0,u*(-1)) for u in [1,-1]])
# Numeric inputs and exactly four parameters assumed. This is one constructed update.
```

Normal,zero,missing-record and changed-protocol probes

Complete initialization/replay probe	Expected answer / action
Normal scalar starts 1 and3	Finalv1.75 and2.25, same MSE.15625, different nonzero-input predictions
Repeat same local seed and same calls	Same four draws within the specified setup; save actual values too
Same seed plus an extra uniform preview draw	Different four-value assignment for seed 17in this example; call history matters
All four symmetry parameters start 1	Gradientsall2→parametersall.8; output 1.28/loss.0784; units still equal
All four symmetry parameters start 0	All gradients 0 under stated zero-ReLU-derivative convention; zero factors block these updates
Input weightsu 1/v−1	Hidden[1, 0]atx 1 and[0, 1]atx−1; distinct outputs demonstrated, not useful learned features proved

Complete initialization/replay probe	Expected answer / action
Missing initial values/code/data/batch order	A seed cannot fill the gap; request/save the actual experiment state
Same wrong output on every replay	Repeatability only; compare independent hand traces, shape/gradient checks and saved inputs
Change initialization and batch order together	Not a one-factor start comparison; restore the same protocol before attributing effects

Predict,restart,run or trace,and retain the full record

1. Predict the three scalar traces and the seeded draw comparison before running or tracing them. The extra-preview program is complete and uses the same generator/method in both cases. Then run or trace the separate four-parameter symmetry update; its zero derivative convention at ReLU0is declared, not proof of differentiability there.

   Save initial arrays, the parameter order, data/split identity, code/environment, rate, batch order, budget, selection policy, all planned runs and final state. Do not select a lucky initialization using final-test outcomes or treat reproducible arithmetic as proof of correctness.

2. On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Keep the evidence ledger

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	

Record	Your evidence
First mismatch and one explanation	
Boundary or missing input and expected response	

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ **Earlier wording, alternatives and picture transcripts**

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: compare starts on a different objective

> Use its supplied minimum,gradient,rate and two-update budget.

Complete new independent starting-value comparison

Use J(a) = (a - 3) squared, gradient 2(a - 3), rate 0.25 and two updates. Restart from a = 1, then restart from a = 5. Calculate both traces and final losses. Explain what can be concluded about these two runs.

First attempt and support record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ **Earlier wording, alternatives and picture transcripts**

Independent case: Transfer the starting-value experiment independently

Attempt this without hints

Use J(a) = (a - 3) squared, gradient 2(a - 3), rate 0.25 and two updates. Restart from a = 1, then restart from a = 5. Calculate both traces and final losses. Explain what can be concluded about these two runs.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

MAIN LESSON · SECTION 11

Keep the actual starting state and full replay record

> The saved experiment should identify every source of change and uncertainty.

Project artifact and evidence table

Add a labelled **initialisation** and reproducibility **evidence** sheet to Image-pattern Network Investigation. Show how you can trace how starting values affect a fixed **training** budget.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run/trace and limits

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

▼ Earlier wording, alternatives and picture transcripts

Evidence for your project

MAIN LESSON · SECTION 16

Finish with one repeated trace and one independent correctness check

> Explain what equal loss and same seed leave undecided.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Trace how starting values affect a fixed **training** budget

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use the mathematical words precisely**

REFERENCE · SECTION 5

Reference · Use the mathematical words precisely

> Connect each term to a value or step in the current example.

Vocabulary and your explanation

Word	Meaning
Initialisation	The procedure that assigns model parameter values before training
Seed	A value used to initialise a pseudorandom generator
Symmetry	An equivalence between parts of a model that can keep their computations identical
Reproducibility record	Saved data settings code and results that let another person check how an experiment was performed

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- For the scalar lesson model, start v = -2 and apply two updates at rate 0.1. Give all three v values and the final **training** MSE.

- Both runs use the same **seed** and generator. Run A assigns the first four draws to its weights. Run B draws a preview number first, then assigns the next four. Must the initial weight arrays match? What should be saved?

- A forward-pass program returns the same wrong answer every run with a fixed **seed**. Has repeatability verified the implementation? Name two stronger checks.

Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the representation and its limits

REFERENCE · SECTION 12

Reference · Check the representation and its limits

> Use after practice;reserve the new independent $J(a)=(a-3)^2$starting-value comparison for the first individual attempt.

Current explanation and response

- Save actual initial parameters, their order, data and split IDs, pre**process**ing rules, model formulas, loss convention, learning rate, batch order, update budget, selection policy, code version, software environment and final checkpoint.

  A **seed** helps replay one source of randomness; it does not specify all of these choices or guarantee identical results across different numerical environments.

- Repeatability alone cannot prove correctness: a program can repeat the same mistake. Preserve hand-calculated checks as well as logs.

  When comparing starts, report every planned run instead of selecting a lucky result using final test outcomes. Next week locates shape and gradient errors.

- A forward-pass program returns the same wrong answer every run with a fixed **seed**. Has repeatability verified the implementation? Name two stronger checks.

> **Two runs use seed 17 but have different final losses. One run made an extra random draw and used a different batch order. Is either difference evidence that the network is broken? Describe a useful investigation.**

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

Identical units receive identical updates

HIDDEN UNIT	BEFORE THE UPDATE	AFTER THE UPDATE
First ReLU unit	u is 1 and a is 1	u is 0.8 and a is 0.8
Second ReLU unit	v is 1 and b is 1	v is 0.8 and b is 0.8

In this separate two-unit example there are no biases. At x = 1, output is 2 and target is 1. Each of the four squared-error partial gradients equals 2. A simultaneous update at rate 0.1 changes every weight from 1 to 0.8 and keeps the units identical.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

> **My complete trace of the worked case**
>
> | | | | | |
> |---|---|---|---|---|
> | | | | | |
> | | | | | |
> | | | | | |
> | | | | | |

▼ **Reference · The earlier route checklist**

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: Reproduce two **training** traces by saving their distinct initial values.

By the end, I can	How I will show it
Trace how starting values affect a fixed training budget	Explain the deciding step, show a trace or test, and state one limit.
Explain a simple hidden-unit symmetry	Explain the deciding step, show a trace or test, and state one limit.
Record enough information to replay and audit a classroom experiment	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ **Extra practice · Predict,check and repair**

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- Reproduce two **training** traces by saving their distinct initial values.
- Which labelled step matches this detail? Save initial values, rate, objective and update count; the same final loss hides different parameter paths.
- 1. Record experiment
- 2. Start from 0
- 3. Start from 4
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Record experiment Save initial values, rate, objective and update count; the same final loss hides different parameter paths.

Initialisation supplies the parameter values before learning begins. The same update rule can follow different paths from different starts.

Here the two paths end on opposite sides of two and have equal squared distance, so equal loss does not imply equal parameters. Save initial values and settings to reproduce this arithmetic, as well as the final result.

Changed-case question and optional hint

- Changed-case practice: Repeat the a = 0 run with identical arithmetic and settings. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. It again reaches 1 then 1.5 in this deterministic toy calculation.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: It again reaches 1 then 1.5 in this deterministic toy calculation. Identical initial state and update rules determine the same arithmetic trace.

Repair and retain the record

Review Regularisation and early stopping (ai-5/16). Goal: Separate a weight penalty from **prediction** error. Short practice: Why should the penalty value not be reported as validation error?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Learning rate experiments (ai-5/14). Goal: Run a controlled comparison of learning rates. Short practice: What should a fair rate comparison hold fixed?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Updating parameters (ai-5/11). Goal: Apply a learning rate to an ordered gradient. Short practice: Why must new loss be recomputed rather than assumed smaller?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

</details>

<details>
<summary>Reference · Six-picture concept journey</summary>

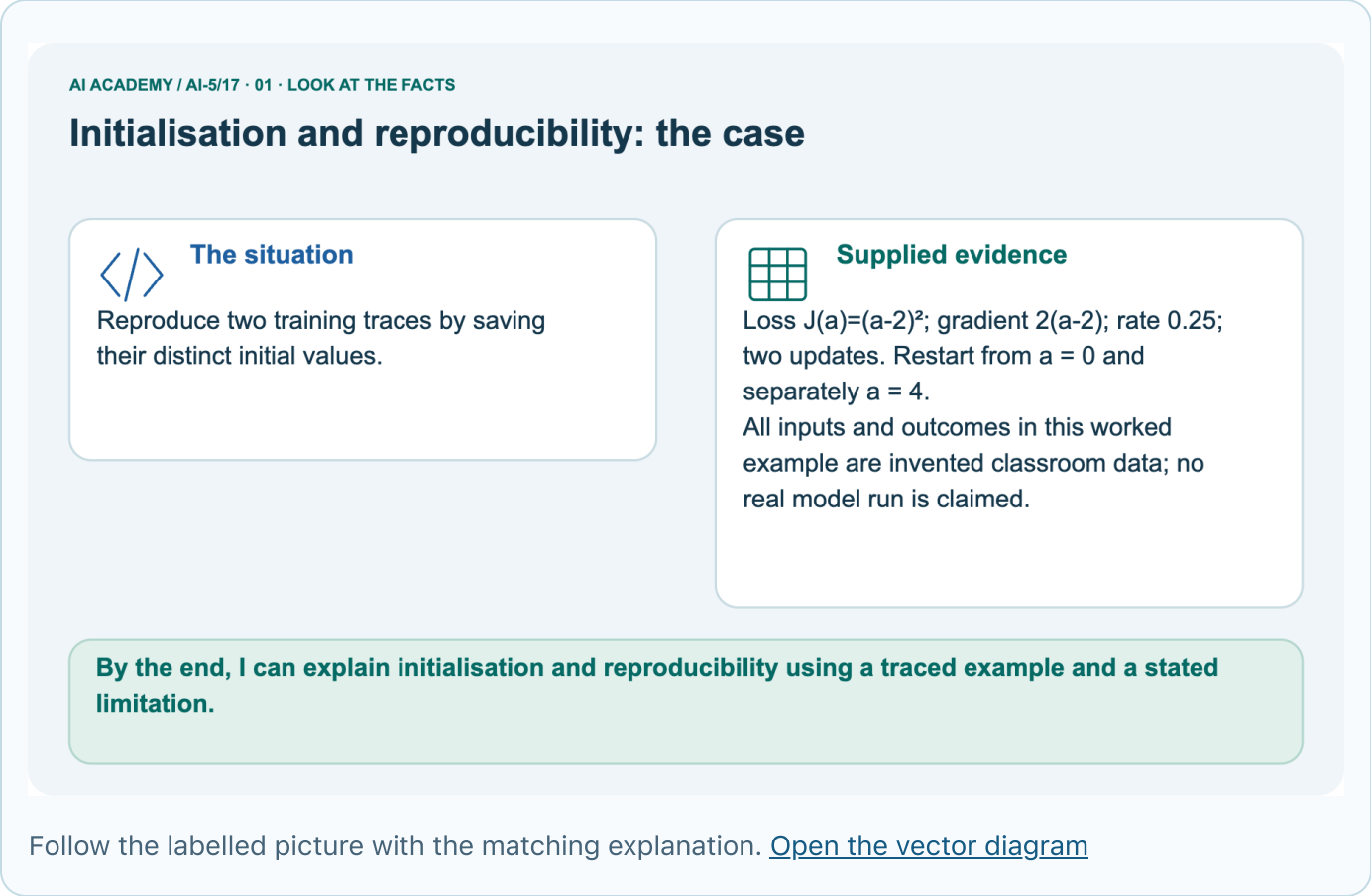
</details>

REFERENCE · SECTION 18

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Initialisation and reproducibility: the case



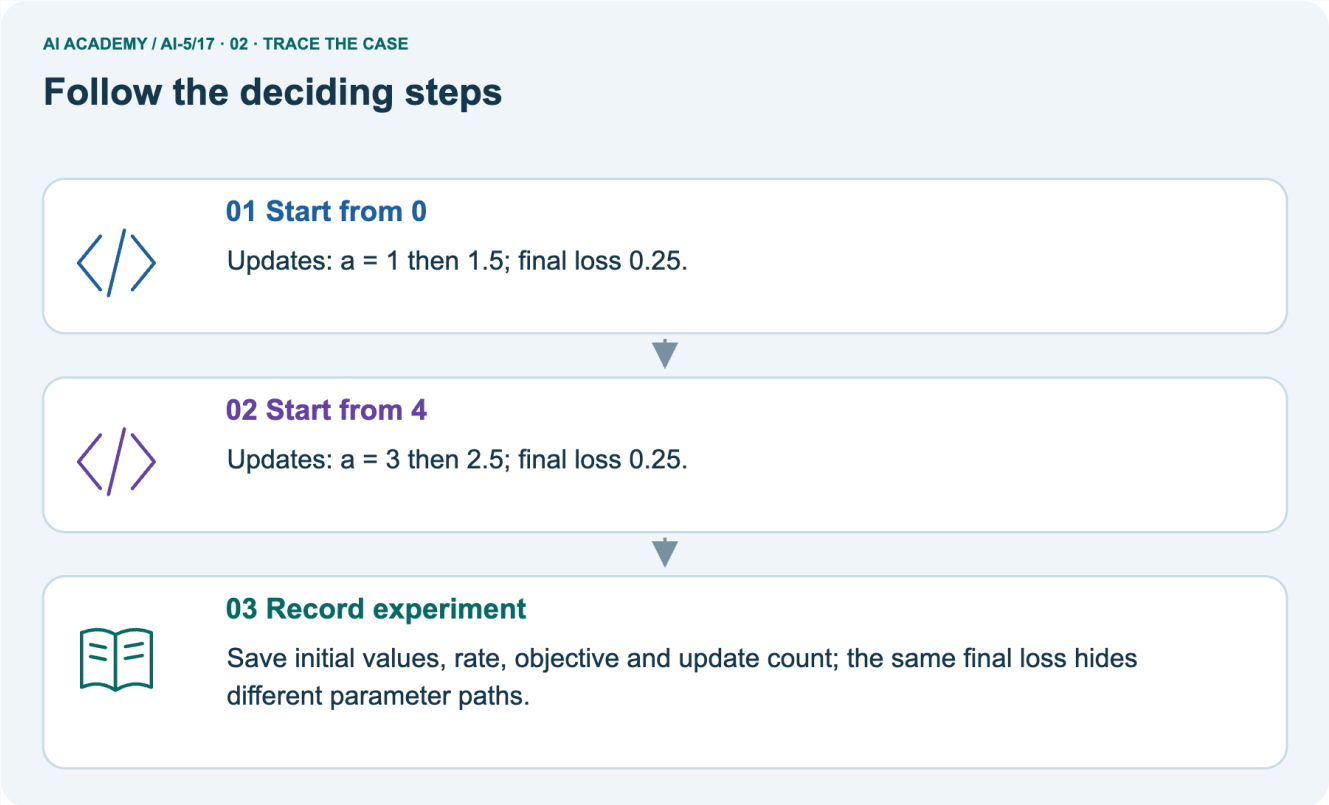
Follow the labelled picture with the matching explanation. Open the vector diagram

Reproduce two **training** traces by saving their distinct initial values. Supplied facts: Loss J(a)=(a-2)²; gradient 2(a-2); rate 0.25; two updates. Restart from a = 0 and separately a = 4.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

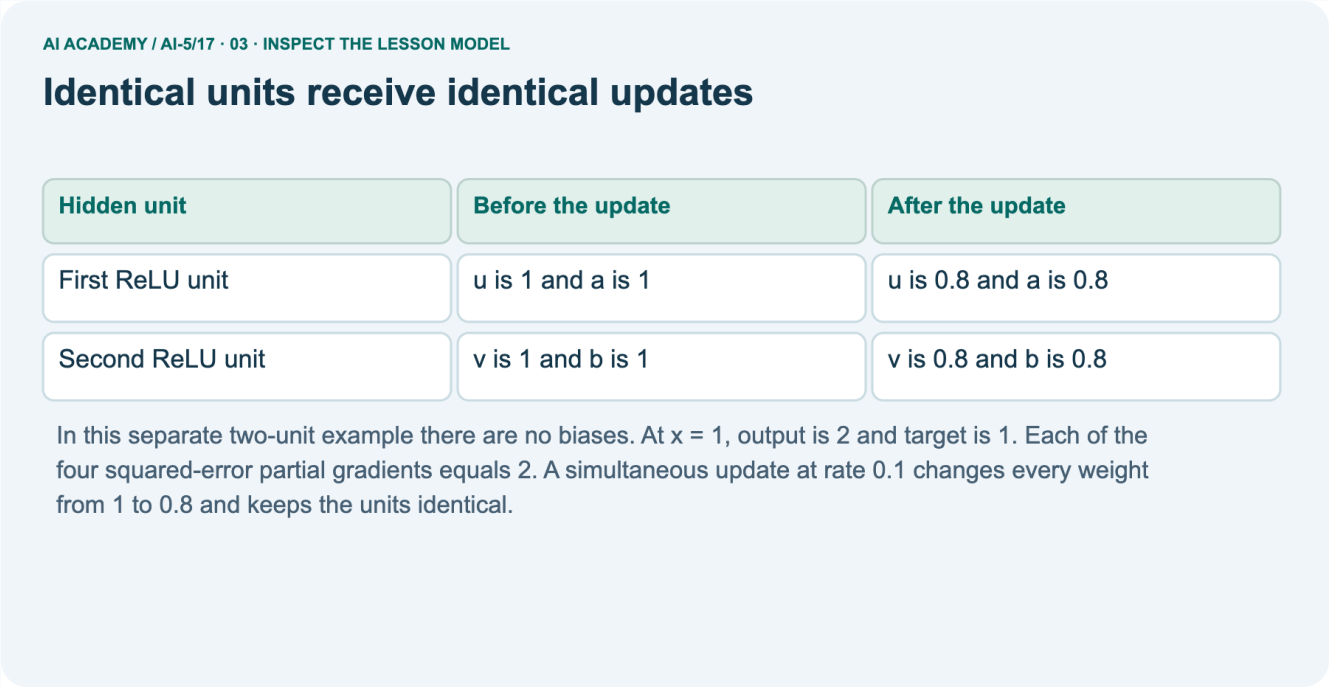


Follow the labelled picture with the matching explanation. Open the vector diagram

Start from 0: Updates: a = 1 then 1.5; final loss 0.25. → Start from 4: Updates: a = 3 then 2.5; final loss 0.25. → Record experiment: Save initial values, rate, objective and update count; the same final loss hides different parameter paths.

Point to each numbered stage and explain which recorded **input** it uses.

03. Identical units receive identical updates

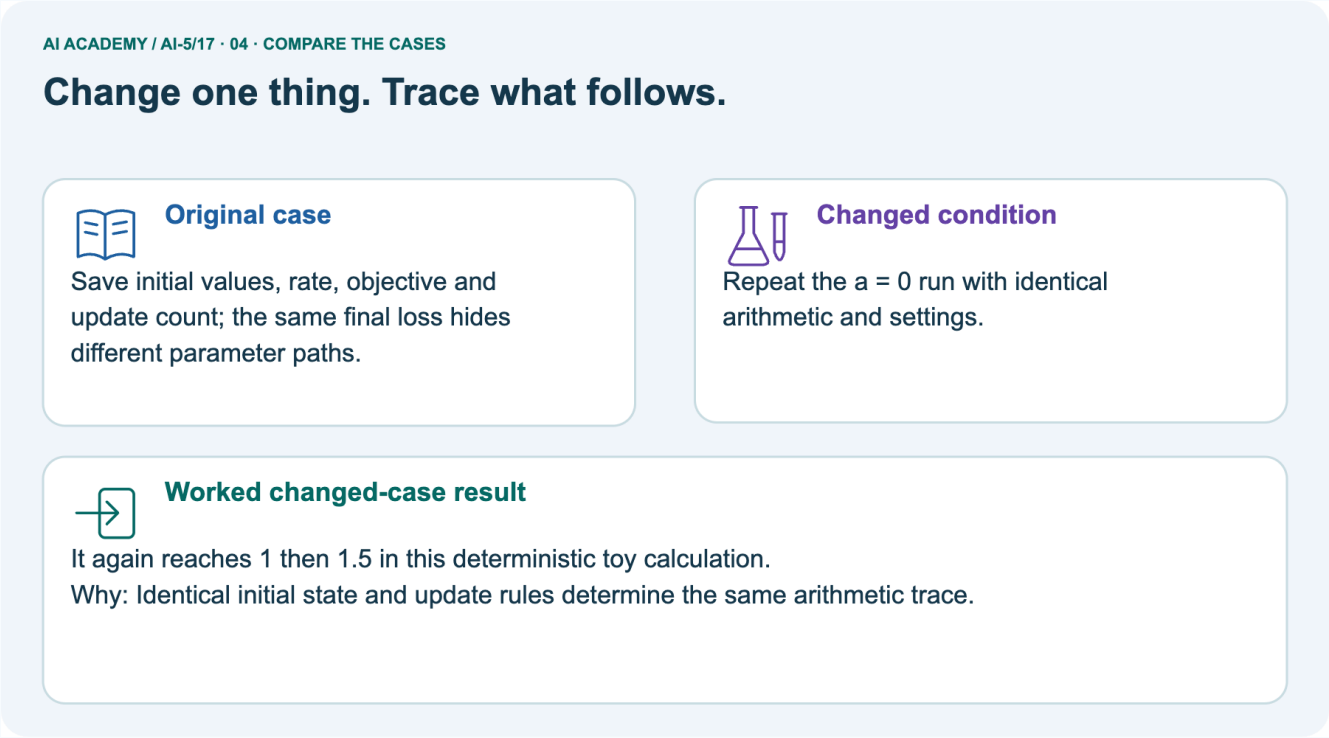


Follow the labelled picture with the matching explanation. Open the vector diagram

Identical units receive identical updates. First ReLU unit | u is 1 and a is 1 | u is 0.8 and a is 0.8. Second ReLU unit | v is 1 and b is 1 | v is 0.8 and b is 0.8

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



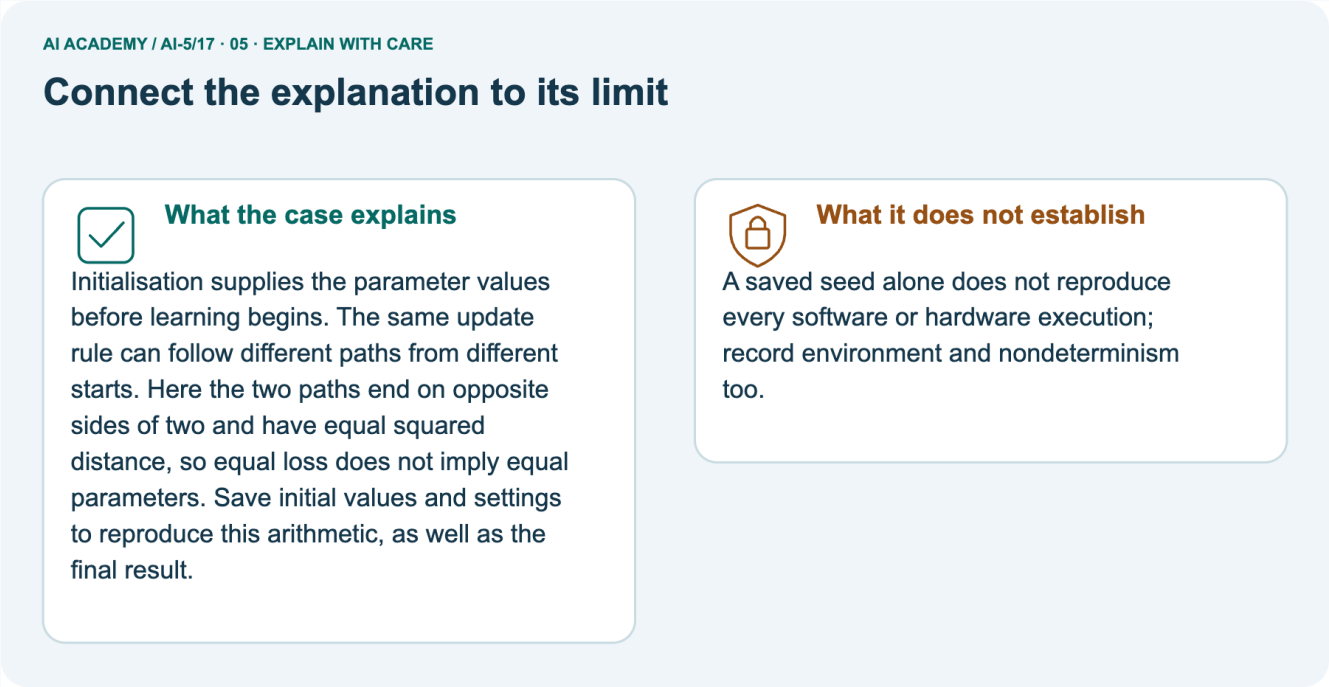
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Save initial values, rate, objective and update count; the same final loss hides different parameter paths. Changed condition: Repeat the a = 0 run with identical arithmetic and settings.

Expected result: It again reaches 1 then 1.5 in this deterministic toy calculation. Reason: Identical initial state and update rules determine the same arithmetic trace.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

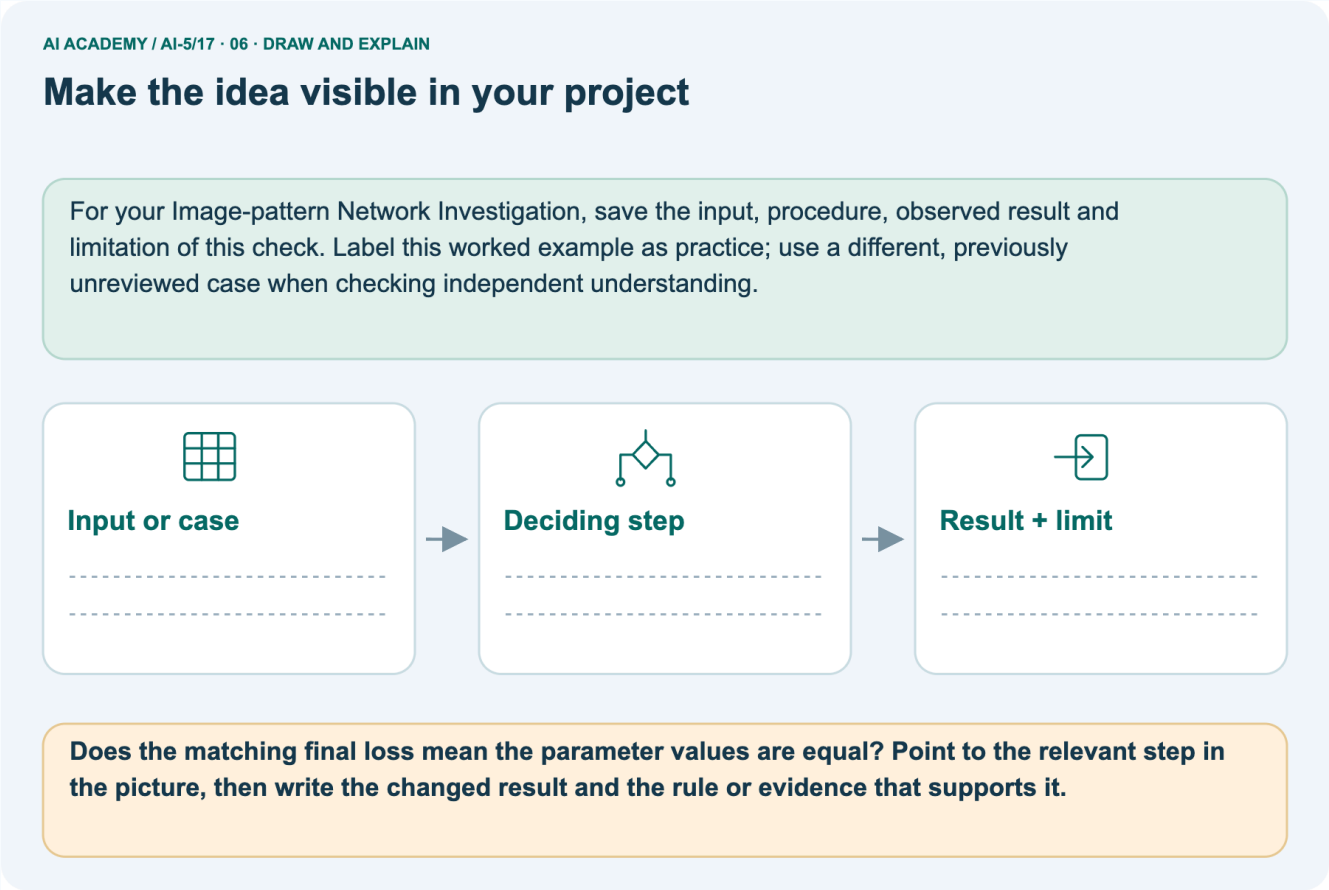
Initialisation supplies the parameter values before learning begins. The same update rule can follow different paths from different starts.

Here the two paths end on opposite sides of two and have equal squared distance, so equal loss does not imply equal parameters. Save initial values and settings to reproduce this arithmetic, as well as the final result.

Limit: A saved **seed** alone does not reproduce every software or hardware execution; record environment and nondeterminism too.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: Does the matching final loss mean the parameter values are equal? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

Earlier wording, alternatives and picture transcripts

01. Initialisation and reproducibility: the case

02. Follow the deciding steps

03. Identical units receive identical updates

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.